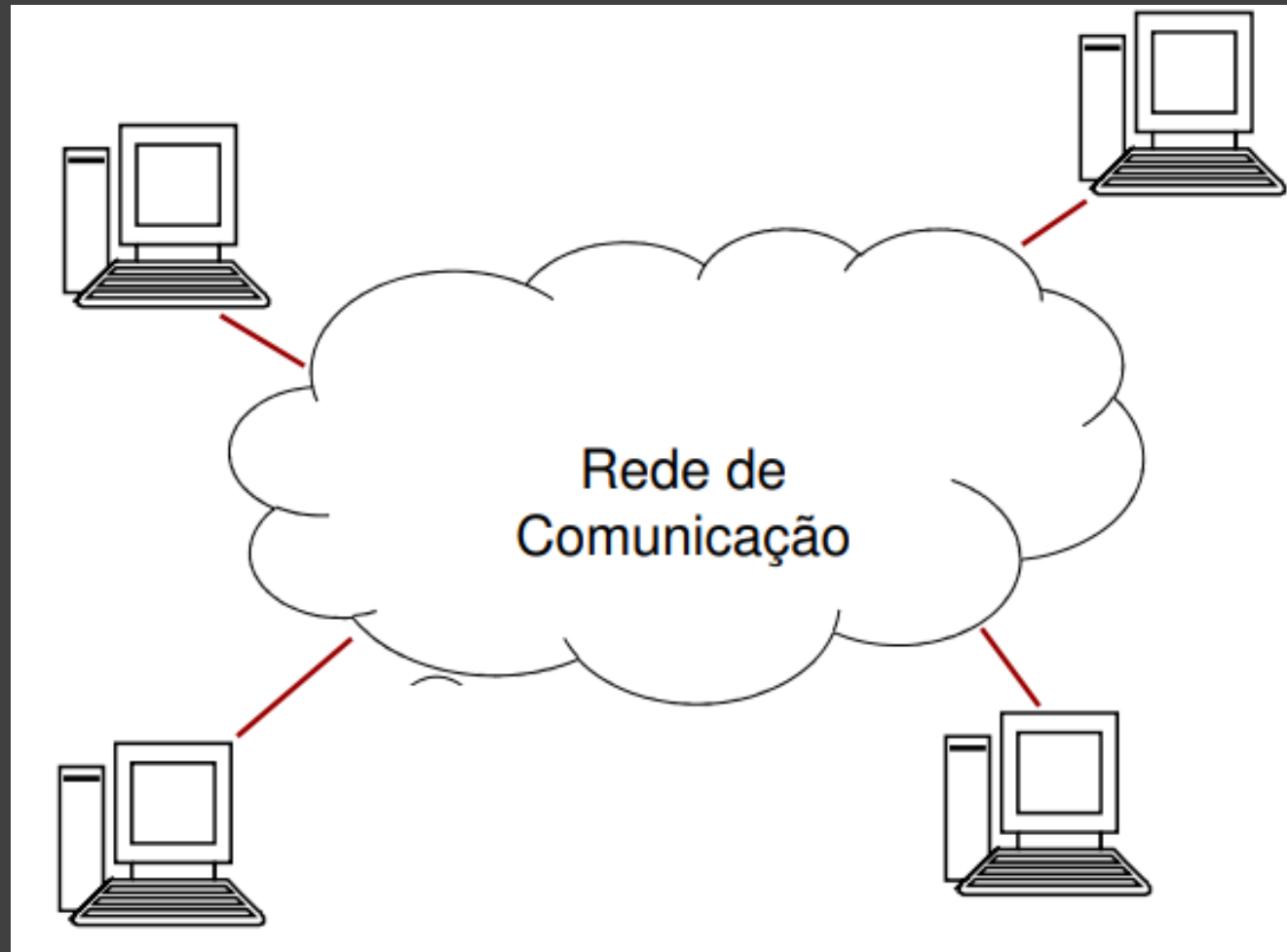


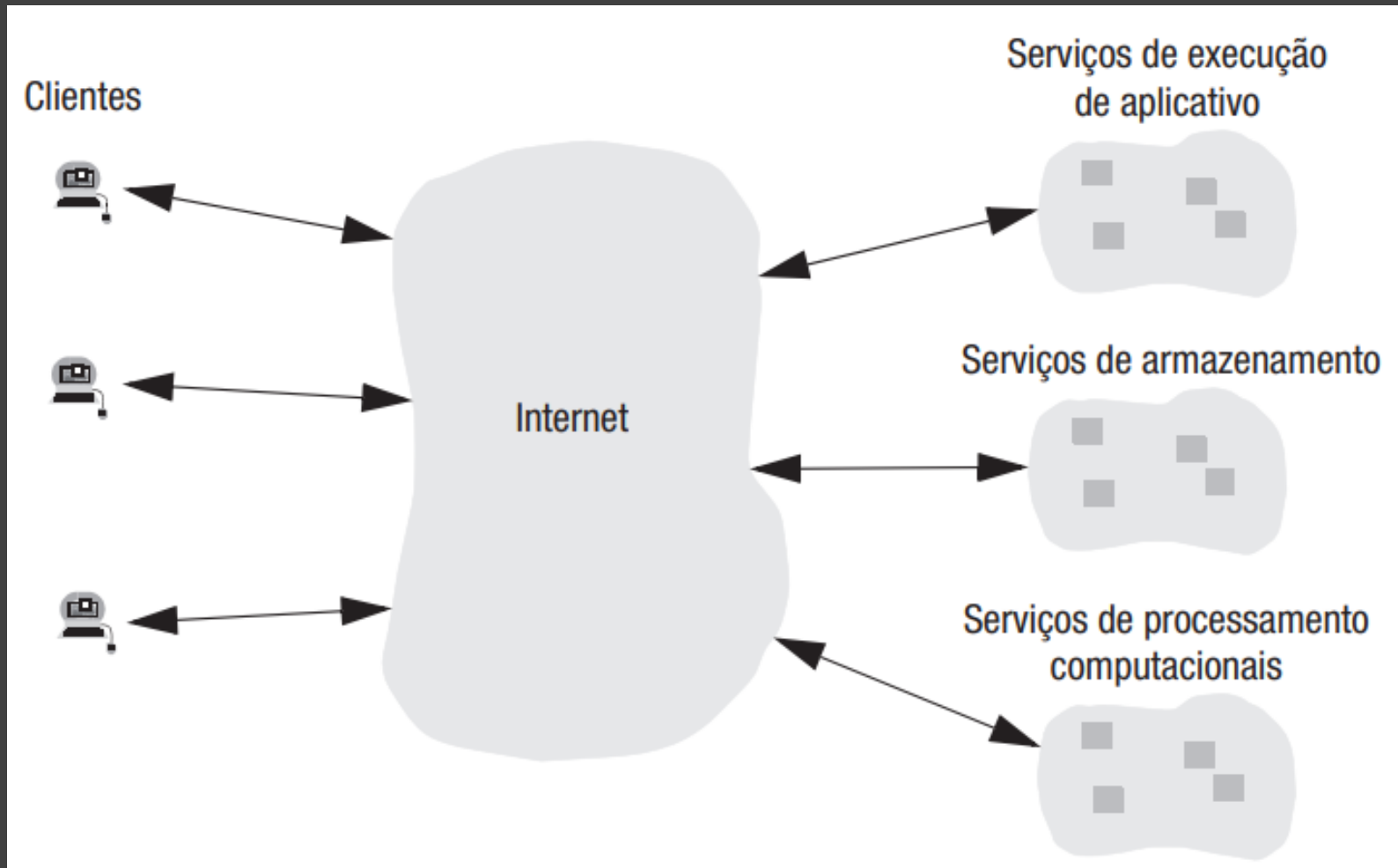
Introdução

PROGRAMAÇÃO DISTRIBUÍDA

O que é um sistema distribuído?



O que é um sistema distribuído?



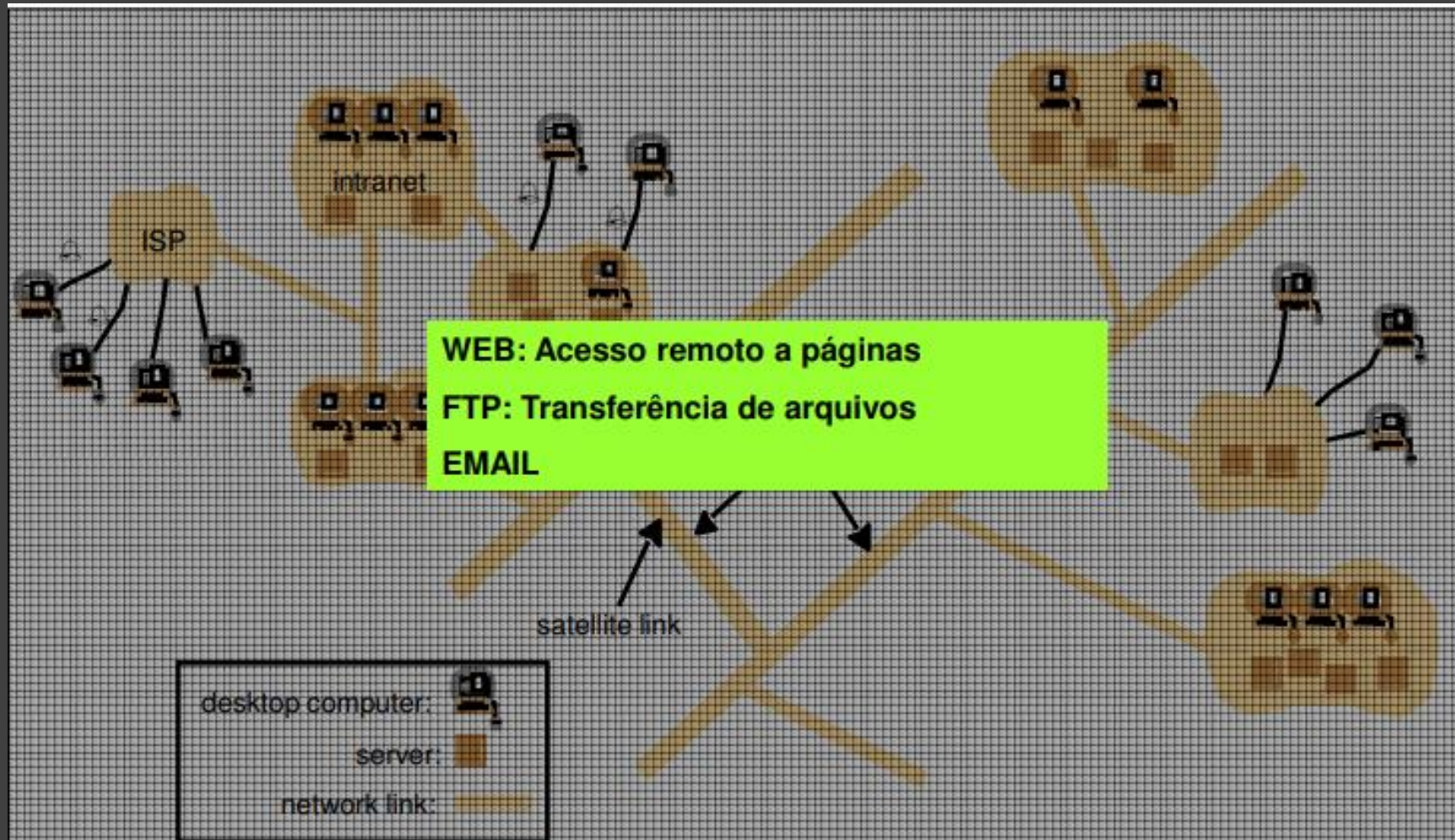
O que é um sistema distribuído?

- Um sistema distribuído é uma coleção de computadores autônomos conectados por uma rede de comunicação que é percebida pelos usuários como um único computador que provê um serviço ou resolve um problema;
- Um sistema distribuído é composto por computadores conectados em rede (hardware e software) que se comunicam e coordenam suas ações somente através do envio de mensagens;

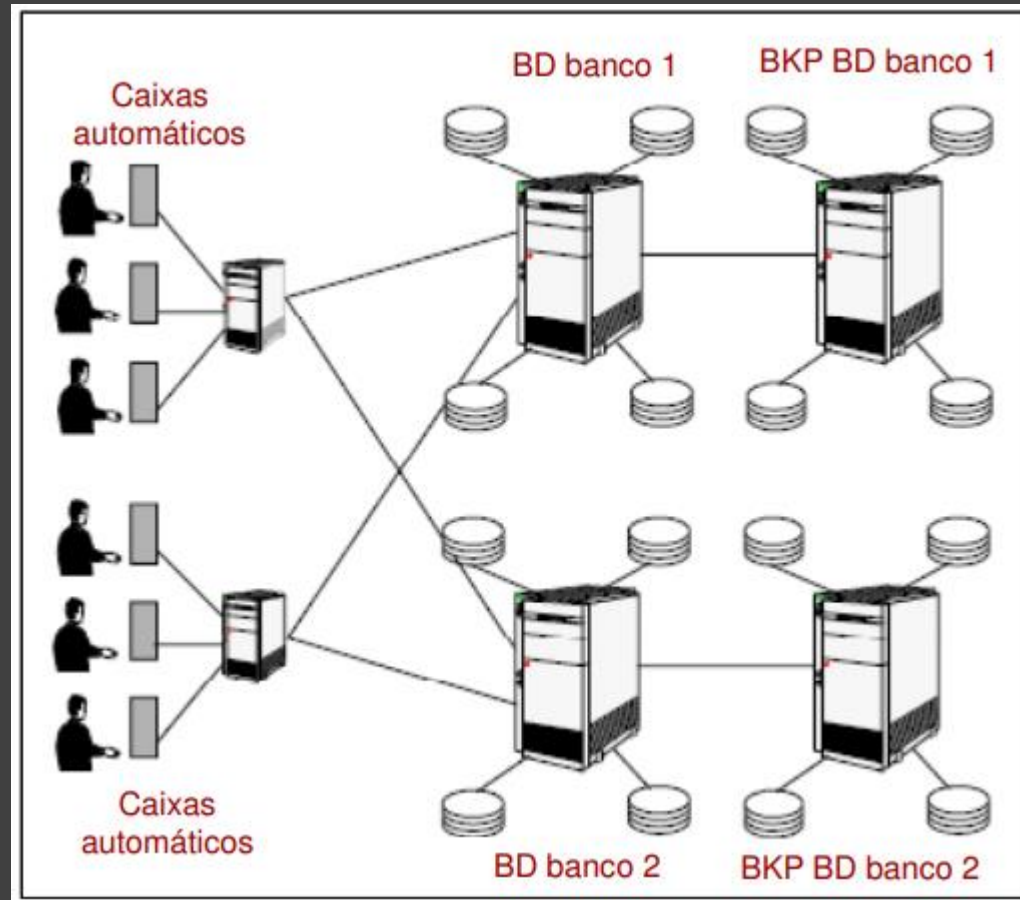
O que é um sistema distribuído?

- Um sistema no qual os componentes de hardware ou software, localizados em computadores interligados em rede, se comunicam e coordenam suas ações apenas enviando mensagens entre si;
- Um sistema distribuído é uma coleção de computadores independentes que parecem ao usuário ser um computador único;

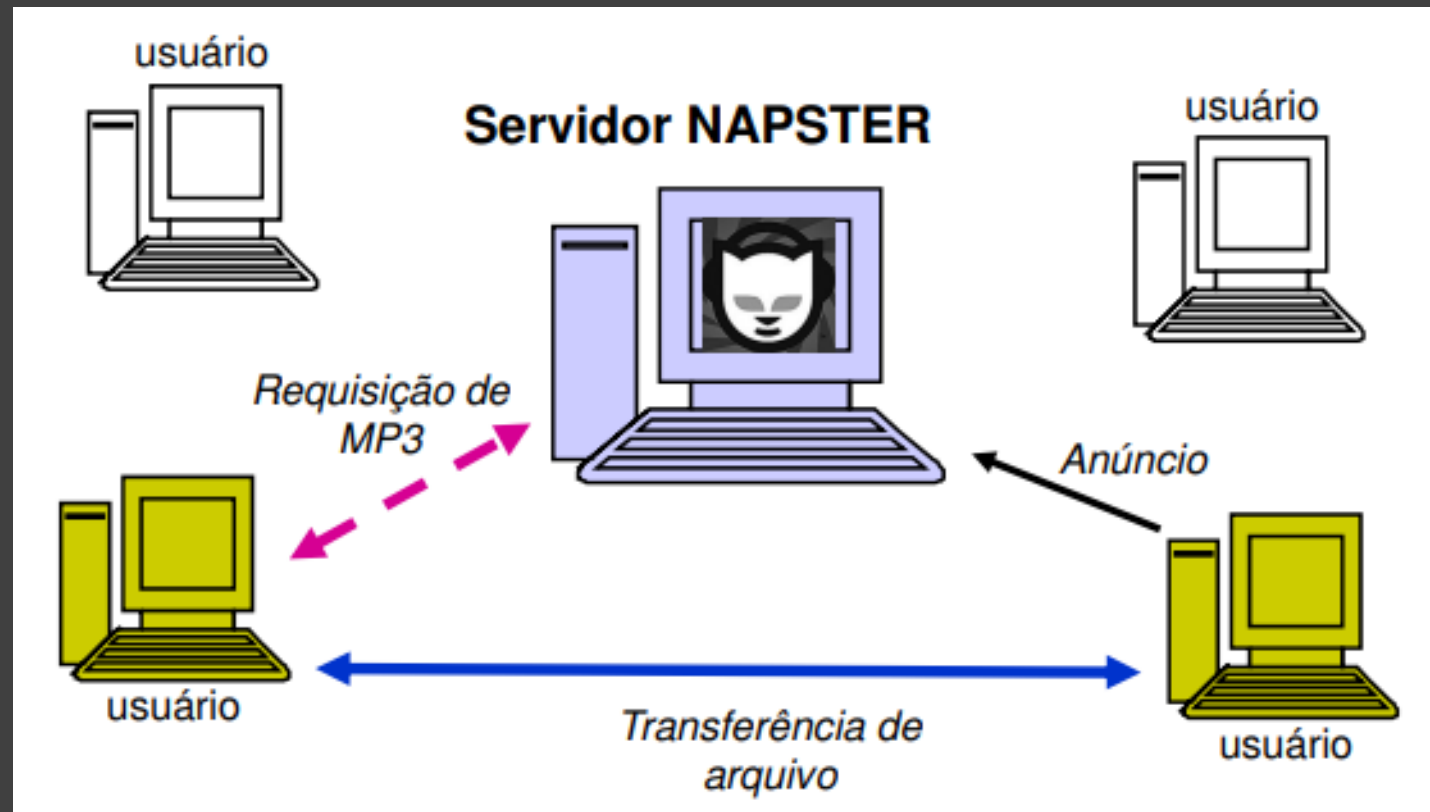
Exemplo : WEB



Exemplo : Bancos



Exemplo : Napster



Exemplos : Outros

- WhatsApp
- Telegram
- Zoom
-

Exemplo : Cluster

- CLUSTERS ou AGRUPAMENTOS
- Máquinas conectadas por rede de alta velocidade
- Necessidade de alto desempenho computacional
- Evitar custo de máquinas de alto desempenho
- Objetivo: compartilhar recursos computacionais
 - De processamento
 - De armazenamento
 - De memória
 - Outros

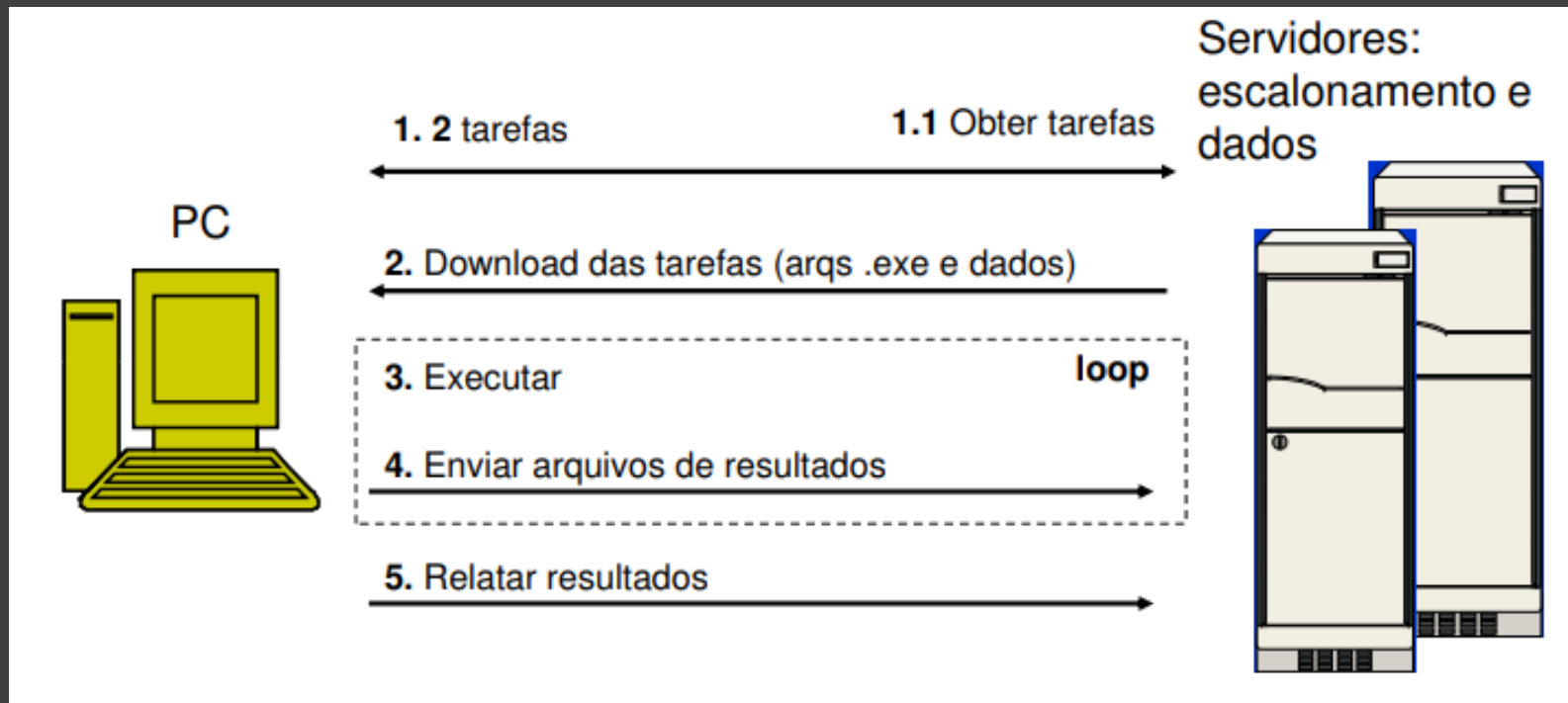
Clusters são utilizados para

- Aumentar a disponibilidade de serviço:
 - se um nodo falha, outro assume;
- Equilibrar carga de trabalho:
 - um ou mais computadores do cluster atuam como distribuidores da carga entre os demais;
- Alto desempenho
 - Para resolver tarefas complexas que podem ser decompostas em sub-tarefas, cada uma rodando num nodo do cluster;
 - Implementação mais comum: LINUX e software livre para implementar paralelismo = Beowulf cluster;

Exemplo : GRID

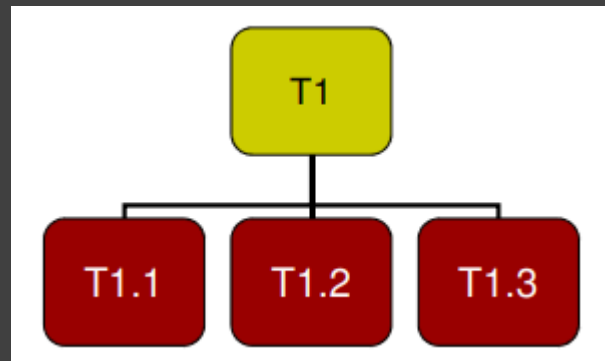
- GRID ou GRADE = as máquinas encontram-se distantes geograficamente;
- É uma cooperativa de máquinas:
 - não pertencem necessariamente a mesma organização;
 - são ditas organizações virtuais;
 - não há administração central dos recursos computacionais;
 - utilização de protocolos/padrões abertos;
- Aplicações:
 - Tarefas complexas decompostas;
 - Reunião dos resultados parciais;

Exemplo : BOING



Aplicações

- Resolução distribuída de problemas;
- Um usuário deseja resolver um problema complexo (ex. renderização de imagens complexas);
- Decomposição de um problema em subproblemas;
- Distribuição dos subproblemas para diferentes processos;
- Um dos processos coleta e monta o resultado final;



Aplicações

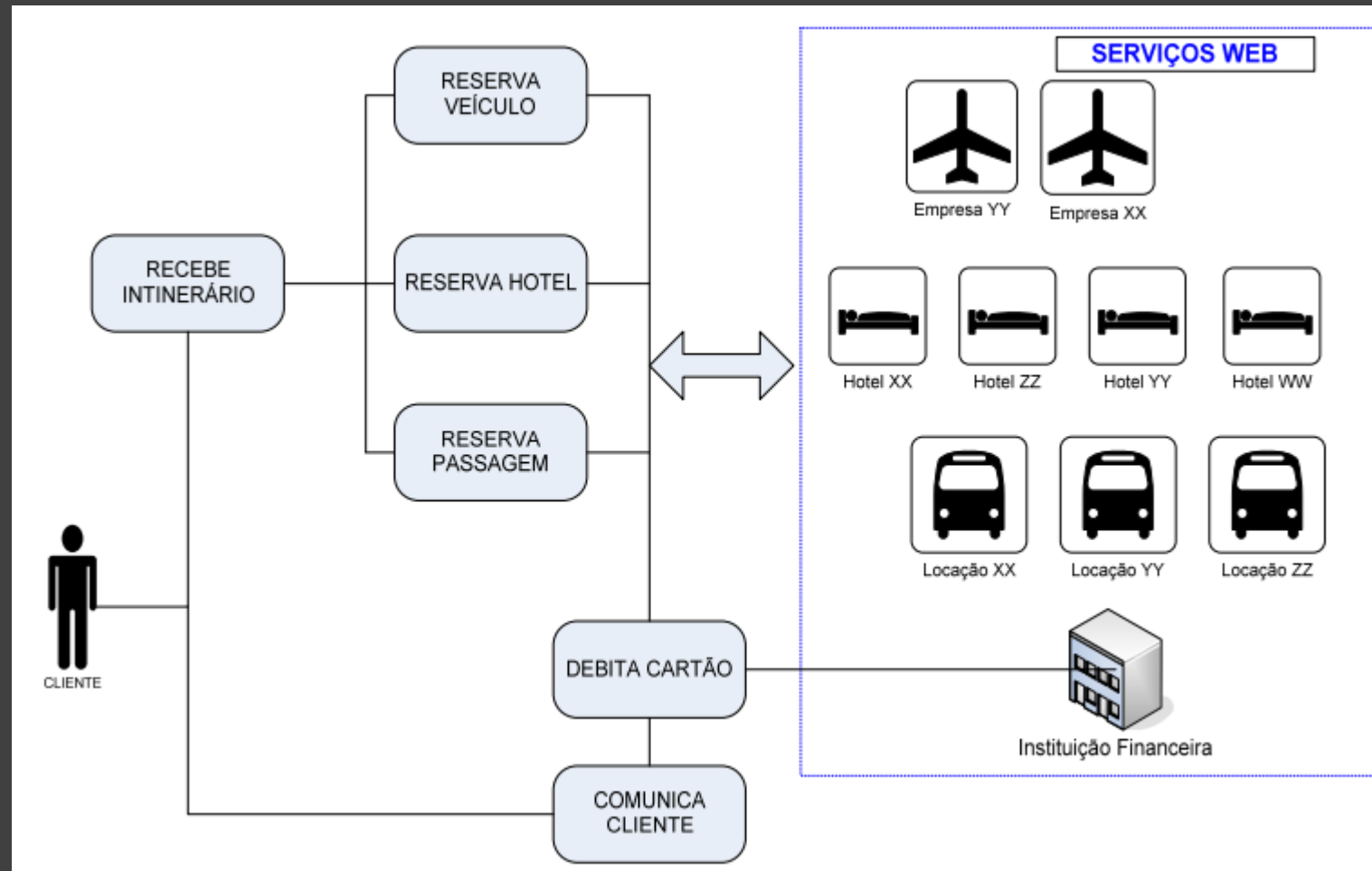


- Renderman da Pixar: <https://renderman.pixar.com/index.html>;
- Filme 2 minutos com 30 frames por segundo, 7.200 minutos de renderização;
- Filme 2 minutos 30 frames por segundo, 36 minutos de renderização;

Aplicações

- Composição de serviços;
- Sistemas multi-agentes/peer-to-peer;
- Usuários com diferentes necessidades utilizam serviços/recursos heterogêneos providos por diferentes desenvolvedores;
- Foco na interação entre agentes (competição ou colaboração);
- Solução surge através da interação;
 - Ex. leilão

Aplicações



Motivações

- Desempenho > sistema centralizado;
- Distribuição natural das aplicações: bancárias, videoconf., etc;
- Robustez: frente a falhas;
- Escalabilidade: sist. pode ser aumentado sem perda de desempenho?
- Compartilhamento de recursos:
 - Impressoras e outros hardwares
 - Arquivos (inclusive páginas Web);
 - CPU;
 - Memória;
 - Espaço em disco;
 - Dados;

Dificuldades

➤ O desenvolvimento de um sistema distribuído é mais complexo do que um centralizado:

- Heterogeneidade;
- Abertos;
- Seguros;
- Escaláveis;
- Confiáveis;
- Concorrência;
- Transparência;

Exemplo

- Baseando-se no exemplo da aplicação bancária mostrada no início destes slides e considerando que:
 - Há acessos múltiplos (concorrentes) à mesma conta;
 - Transações bancárias podem envolver contas armazenadas em diferentes bancos de dados (BD);
 - Não interessa ao cliente se os dados de sua conta estão armazenados no BD1 ou BD2
- Pode-se concluir que as características importantes para este tipo de sistema são... (> próximo)

Exemplo

- Acessos múltiplos (concorrentes) à mesma conta: vários usuários acessam a mesma conta ou um sistema do banco (emissão de relatórios) acessa ao mesmo tempo em que usuários realizam transações;
- Transações bancárias podem envolver contas armazenadas em diferentes bancos ou localidades;
- O serviço deve ser transparente ao usuário – não interessa se os dados de sua conta estão armazenados no banco 1 ou banco 2, a aplicação não deve em nenhum momento deixar transparecer aspectos internos do sistema;
- O serviço deve ser confiável e seguro;
- Bases de dados replicadas devem ser consistentes;
- Tolerância a falhas;

Heterogeneidade

- Lidar com diferentes:
 - Redes;
 - Hardwares (computadores);
 - Sistemas operacionais;
 - Linguagens de programação;
- Além de resolver os problemas de heterogeneidade, o middleware define um modelo computacional uniforme para a programação de aplicações distribuídas;

Aberturas

- O grau de abertura de um SD depende de:
 - Facilidade de adicionar novos recursos:
 - De hardware;
 - De software: novos serviços ou re-implementação de antigos;
 - SD abertos baseiam-se em:
 - Mecanismos de comunicação uniformes;
 - ✓ Especificados por organismos tais como OMG e IETF;
 - ✓ Ex. CORBA (OMG);
- Publicação de interfaces de serviços: para esconder a heterogeneidade de implementação dos serviços, as implementações devem aderir a uma forma de publicar as interfaces dos serviços;

Segurança

- “selar” informações importantes enviadas através da rede (ex. criptografia de número de cartão de crédito);
- Identidade: garantir a autenticidade da fonte:
 - Ex. um médico deseja acessar dados de pacientes de forma remota. É preciso assegurar que o usuário é realmente o médico;
 - Ex. um usuário deseja comprar um livro pela Internet. É preciso assegurar a identidade da loja virtual - que realmente é uma loja e não vai fazer mal uso do cartão;
- Problemas :
 - Indisponibilidade de serviço devido a ataques;
 - Segurança de código móvel (ex. .exe anexados a emails, applets);

Tratamento de Falhas

- Falhas atingem parcialmente um SD;
- Detecção:
 - Mensagens corrompidas => checksum;
 - Crash de um servidor => nunca é certo se o servidor caiu, se a falha está no link ou se a rede está sobrecarregada;
- Esconder falhas:
 - Se uma mensagem não chega no destino => retransmissão;
 - Dados podem ser escritos em dois discos, se um está danificado, utiliza-se a cópia;

Tratamento de Falhas

➤ Tolerância a falhas:

- Exemplo: quando um navegador da Internet não consegue acessar um servidor ele não prende o usuário indefinidamente;

➤ Como tornar um sistema tolerante a falhas?

- Redundância pode tornar sistema tolerante a falhas;
- Replicação de dados;
- No serviço DNS cada entrada é replicada em pelo menos dois servidores;
- Explorar rotas alternativas na Internet;

Resumo

- Sistemas distribuídos são úteis para:
 - Melhorar desempenho;
 - Melhor adaptados a distribuição natural dos elementos;
- Porém são complexos:
 - São inerentemente concorrentes;
 - Devem ser confiáveis;
 - Devem ser transparentes;
 - Funcionar em plataformas heterogêneas;
- Middlewares resolvem parte destes problemas;

DÚVIDAS, PERGUNTAS, QUESTIONAMENTOS

